

Three different machines. Three different sets of problems. Each time.

Docker fixes this. Your setup behaves the same on every machine. Period. No more “but it worked on my laptop” debates.

It also keeps things contained. The Ollama process inside a container cannot accidentally break other things on the same machine. You set memory limits, network rules, security policies in one place.

Once you have a Docker image, you are basically ready for Kubernetes whenever you need it. Scaling, rolling updates, health checks. All standard stuff.

You can also run multiple Ollama containers on one machine. Each with a different model. Each with different limits. This sounds fancy but it is genuinely useful. My team had three people testing different models last week. Same machine. No fights.

And here is the simplest reason of all. Your team probably already lives in Docker. Adding Ollama is just one more service in something they already understand.

Building this thing step by step

Okay, let us actually do it. I am assuming Linux with Docker installed. Windows works through WSL2. macOS works with small changes.

Step 1: Install Docker and the NVIDIA Container Toolkit

If you have a graphics card, you want Docker to use it. Otherwise that GPU is just an expensive paperweight.

```
curl -fsSL https://nvidia.github.io/libnvidia-container/
gpgkey | \
  sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-
  toolkit-keyring.gpg
```

```
curl -s -L https://nvidia.github.io/libnvidia-container/
stable/deb/nvidia-container-toolkit.list | \
  sed 's#deb https://#deb [signed-by=/usr/share/keyrings/
  nvidia-container-toolkit-keyring.gpg] https://#g' | \
  sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.
  list
```

```
sudo apt update
sudo apt install -y nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

Test it: `docker run --rm --gpus all nvidia/cuda:12.4.0-base-ubuntu22.04 nvidia-smi`. Your GPU should show up.

Step 2: Run Ollama in Docker

```
docker run -d \
```

```
--gpus=all \
-v ollama_data:/root/.ollama \
-p 11434:11434 \
--name ollama \
--restart unless-stopped \
ollama/ollama
```

A small thing that will save your life later. That `-v ollama_data:/root/.ollama` part. It saves your downloaded models. Forget this once and you will redownload a 40GB model. Do it twice and you will set a reminder. I learned this on a slow office connection. It was not a good day.

No graphics card? Drop the `--gpus=all` flag. Smaller models like Phi-3 or Llama 3.2 3B run fine on CPU.

Step 3: Pull a model and try it

```
docker exec -it ollama ollama pull llama3.1:8b
docker exec -it ollama ollama run llama3.1:8b
```

Done. You have a working AI model. Test the API:

```
curl http://localhost:11434/api/generate -d '{
  "model": "llama3.1:8b",
  "prompt": "Explain Kubernetes in one paragraph.",
  "stream": false
}'
```

Step 4: Move to Docker Compose

Once you stop playing around, please use Compose. Three months from now you will not remember which flags you used. I promise.

```
version: '3.8'

services:
  ollama:
    image: ollama/ollama:latest
    container_name: ollama
    restart: unless-stopped
    ports:
      - "11434:11434"
    volumes:
      - ollama_data:/root/.ollama
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]
    open-webui:
      image: ghcr.io/open-webui/open-webui:main
```